A PROJECT REPORT

ON

ADVANCE JAVA LAB

AETHERIS
Digital Environment Twin System

Submitted in partial fulfillment of the requirement for the award of Degree of Bachelor of Technology in AI & Data Science

Submitted to:

Rajasthan Technical University, Kota (Raj.)

Under the Guidance of
Er. Ram Babu Buri

Submitted By:

Devansh Joshi	(24EARAD039)
Devansh Mittal	(24EARAD041)
Dhruv Jain	(24EARAD043)
Garv Kumar	(24EARAD049)
Ishita Sinha	(24EARAD063)

Session: 2026-27

Department of Artificial Intelligence & Data Science
ARYA COLLEGE OF ENGINEERING & I.T.
SP-42, RIICO INDUSTRIAL AREA, KUKAS, JAIPUR

ARYA College of Engineering & Information Technology
(Approved by AICTE & Affiliated to RTU, Kota)
SP-42, RIICO Industrial Area, Delhi Road, Kukas, Jaipur - 302028 (Rajasthan) India
Tel. No. : 0141-6604555
Toll Free : 1800-266-2000
Email Id : info@aryacollege.in
Website : www.aryacollege.in
Estd. Yr. 2000
ACCREDITED BY: NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL | NBA NATIONAL BOARD OF ACCREDITATION FOR ECE & IT

Department of Artificial Intelligence & Data Science

CERTIFICATE OF APPROVAL

The Project Report entitled **"AETHERIS – Digital Environment Twin System"** submitted by "Devansh Joshi (24EARAD039), Devansh Mittal (24EARAD041), Dhruv Jain (24EARAD043), Garv Kumar (24EARAD049) and Ishita Sinha (24EARAD063)" has been examined by us and is hereby approved for carrying out the project leading to the award of the degree **"Bachelor of Technology in AI & Data Science".** By this approval the undersigned does not necessarily endorse or approve any statement made, opinion expressed or conclusion drawn therein, but approve the pursuance of the project only for the above-mentioned purpose.

Mentor
Er. Santosh Kumar
Associate Professor

ARYA College of Engineering & Information Technology
(Approved by AICTE & Affiliated to RTU, Kota)
SP-42, RIICO Industrial Area, Delhi Road, Kukas, Jaipur - 302028 (Rajasthan) India
Tel. No. : 0141-6604555
Toll Free : 1800-266-2000
Email Id : info@aryacollege.in
Website : www.aryacollege.in
Estd. Yr. 2000
ACCREDITED BY: NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL | NBA NATIONAL BOARD OF ACCREDITATION FOR ECE & IT

Department of Artificial Intelligence & Data Science

CERTIFICATE

This is to certify that the work embodied in this Project entitled **"AETHERIS – Digital Environment Twin System"** being submitted by **"Devansh Joshi (24EARAD039), Devansh Mittal (24EARAD041), Dhruv Jain (24EARAD043), Garv Kumar (24EARAD049) and Ishita Sinha (24EARAD063)"** in partial fulfillment of the requirement for the award of **"Bachelor of Technology in AI & Data Science"** to **Rajasthan Technical University, Kota (Raj.)** during the academic year 2026-27 is a record of bonafide work, carried out by them under the supervision and guidance of the Project Guide in the Department of Artificial Intelligence & Data Science, Arya College of Engineering & Information Technology, Jaipur.

Project Guide
Er. Ram Babu Buri

Head of Department
Dr. Ashok Kumar Kajla

ARYA College of Engineering & Information Technology
(Approved by AICTE & Affiliated to RTU, Kota)
SP-42, RIICO Industrial Area, Delhi Road, Kukas, Jaipur - 302028 (Rajasthan) India
Tel. No. : 0141-6604555
Toll Free : 1800-266-2000
Email Id : info@aryacollege.in
Website : www.aryacollege.in
Estd. Yr. 2000
ACCREDITED BY: NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL | NBA NATIONAL BOARD OF ACCREDITATION FOR ECE & IT

Department of Artificial Intelligence & Data Science

DECLARATION

We **"Devansh Joshi, Devansh Mittal, Dhruv Jain, Garv Kumar, and Ishita Sinha",** students of **Bachelor of Technology in AI & Data Science, session 2026-27, Arya College of Engineering & Information Technology, Jaipur,** hereby declare that the work presented in this Project entitled **"AETHERIS – Digital Environment Twin System"** is the outcome of our own work, is bonafide and correct to the best of our knowledge and this work has been carried out taking care of Engineering Ethics. The work presented does not infringe any patented work and has not been submitted to any other University or anywhere else for the award of any degree or any professional diploma.

Devansh Joshi (24EARAD039)
Devansh Mittal (24EARAD041)
Dhruv Jain (24EARAD043)
Garv Kumar (24EARAD049)
Ishita Sinha (24EARAD063)

Date: 08-10-2026

ABSTRACT

AETHERIS — Digital Environment Twin System is a web-based application designed to visualize environmental conditions and simplify the monitoring of air quality and microclimate data. It provides an interactive digital representation of environmental sensor stations and displays parameters such as **Air Quality Index (AQI), temperature, humidity, particulate matter, and wind** through a user-friendly dashboard.

The application uses **Java 17, Spring Boot 3.3.5, Spring Data JPA, MySQL, HTML5, CSS3, JavaScript, and HTML5 Canvas**. Its features include user authentication, role-based access control, station management, GPS-based nearest-station identification, live and historical telemetry, and environmental alerts. It also provides a **Clean Air Score (0–100), interactive 2.5D Digital Twin visualization,** hazard simulations, automated telemetry updates, and environmental reports. **The Haversine formula** calculates the distance and bearing to the nearest registered station.

AETHERIS is an **academic prototype** that primarily uses simulated environmental telemetry rather than guaranteed readings from physical sensors. The project demonstrates how web technologies, geospatial calculations, database management, and interactive visualization can make environmental information easy to understand.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide, **Er. Ram Babu Buri,** for his valuable guidance, support, and encouragement throughout the development of our project, **"AETHERIS — Digital Environment Twin System."** His suggestions and technical guidance helped us improve our understanding and complete this project successfully.

We are thankful to the **Department of Artificial Intelligence & Data Science and Arya College of Engineering & Information Technology, Jaipur,** for providing us with the necessary facilities and learning environment to carry out this project as part of the **Advance Java Lab.**

We also express our gratitude to our faculty members and everyone who supported us with their valuable suggestions and feedback. Finally, we thank all our team members for their cooperation, dedication, and teamwork throughout the project.

This project has provided us with practical experience in Java development, database management, web technologies, and software system design.

TABLE OF CONTENTS

Section	Particulars	Page No.
1.	**INTRODUCTION**	**1**
1.1	Overview	1
1.2	Problem Statement	1
1.3	Objectives	2
1.4	Applications	3
2.	**SOFTWARE & TECHNOLOGY ARCHITECTURE**	**4**
2.1	Technologies Used	4
2.2	Technology Specifications	5
2.3	Team Structure & Module Ownership	5
3.	**DESIGN & METHODOLOGY**	**7**
3.1	System Architecture	7
3.2	Geodesic Pairing Algorithm (Haversine)	8
3.3	AQI Verdicts and Clean Air Score	8
3.4	Edge-Case Handling	9
3.5	Authentication & Password Hashing	10
3.6	Hazard Injection Engine (Scenario State Machine)	11
3.7	Database Design	11
3.8	REST API Reference	13

4.	**RESULT AND DISCUSSION**	**14**
4.1	System Performance	14
4.2	Project Screenshots	14
4.3	Algorithm & Security Reliability	18
4.4	Limitations Observed	18
5.	**CONCLUSION AND RECOMMENDATION**	**20**
5.1	Conclusion	20
5.2	Key Achievements	20
5.3	Recommendations for Future Scope	20
6.	**APPENDICES**	22
A	Appendix A: System Diagrams	22
B	Appendix B: Core Source Code (Java)	25
7.	**REFERENCES**	**29**

1. INTRODUCTION

1.1 Overview

AETHERIS is a full-stack, real-time **Digital Environment Twin System** developed for the Advance Java Lab. It is designed for monitoring urban microclimates, industrial particulate dispersion and ecological bio-reserves. A digital twin is a live virtual copy of a physical environment: the sensor stations of a city, a factory zone or a forest reserve are mirrored on screen, and their air quality, temperature, humidity, noise, UV and wind values change in real time.

The system couples an interactive **HTML5 Canvas 2.5D visual digital twin** with a **Java Spring Boot 3.3.5 backend**, a **MySQL database** (with an automatic in-memory H2 fallback), secure token-based authentication with role-based access control, and **device geolocation pairing** with an on-screen permission popup. The product is delivered as three experiences in one: a Product Showcase landing page, a Citizen and Business friendly dashboard with a Clean Air Score, and an Advanced Digital Twin Engineering core.

The backend is built with Core and Advance Java technologies: Java 17, Spring Boot, Spring Data JPA, REST controllers, DTOs, and a scheduled simulation heartbeat.

1.2 Problem Statement

Environmental data in most cities and campuses is difficult to understand and slow to reach the people who need it. Raw numbers such as PM2.5 or AQI mean little to a citizen, a school or a small business owner, while industrial and municipal operators have no safe way to test how their area would behave during a heatwave or an emission event. There is a need for a smart and easy to use system that can:

- Show live environmental data from many sensor stations on a single, visual and interactive digital twin.

- Convert technical readings into a simple Clean Air Score and plain-English health advice for ordinary citizens.
- Link a user's physical device to the nearest sensor station using GPS, distance and compass bearing.
- Allow operators to inject "what-if" hazard scenarios (heatwave, industrial emission, rain) and observe the response safely.
- Protect the system with authentication, hashed passwords and role-based access for administrators, operators and researchers.
- Generate a formal, audit-ready environmental compliance report with one click.

1.3 Objectives

The main objectives of this project are:

- Develop a Java Spring Boot 3.3.5 REST backend with a clean layered architecture and Spring Data JPA.
- Design a MySQL database with four core tables (users, stations, telemetry records, environmental alerts) and an intelligent fallback to in-memory H2.
- Build a 2.5D HTML5 Canvas digital twin with vector wind currents, PM2.5 / PM10 dispersion and thermal gradients, in three visualization modes.
- Implement a Haversine based GPS pairing radar that gives distance, compass bearing and the nearest station.
- Create a Citizen Dashboard with an animated Clean Air Score (0 to 100) and dynamic health verdicts.
- Implement salted SHA-256 authentication with ADMIN, OPERATOR and RESEARCHER roles.
- Design a Digital Twin "What-If" hazard injection engine and an automated background simulation heartbeat using @Scheduled.

- Provide a guided product tour and a certified executive audit report (print / PDF) conforming to ISO 14001:2015 style formatting.

1.4 Applications

- **Smart Municipalities:** city-wide air quality and microclimate monitoring and emergency response planning.
- **Heavy Industry:** particulate dispersion monitoring and automated ESG / compliance reporting.
- **Ecological Bio-Reserves:** protection of forests, wetlands and wildlife areas through continuous monitoring.
- **Corporate Campuses:** HVAC and ventilation planning, and healthy-workplace dashboards.
- **Educational Institutions:** Environmental monitoring across college campuses to help students and administrators understand local air quality and microclimate conditions.
- **Disaster Management:** Simulation of heatwaves and industrial emission scenarios to support preliminary environmental risk assessment and response planning.

2. SOFTWARE & TECHNOLOGY ARCHITECTURE

2.1 Technologies Used

Component	Technology	Role in System
Programming Language	Java 17	Core business logic, entities, services and controllers
Backend Framework	Spring Boot 3.3.5	REST API, dependency injection and scheduled tasks
Data Access	Spring Data JPA	Repositories and entity mapping for all tables
Database	MySQL (fallback: H2)	Permanent storage of users, stations, telemetry and alerts
Build Tool	Maven (Maven Wrapper)	Build and run with only Java 17 installed
User Interface	HTML5, CSS3, JavaScript	Landing page, citizen dashboard and engineering core
Visualization	HTML5 Canvas	2.5D terrain engine, particle system and trend charts
Security	Salted SHA-256, token auth	Password protection and role-based access control
Geolocation	Browser GPS API, Haversine	Device pairing, distance and bearing calculation

2.2 Technology Specifications

- **Language:** Java 17, an object oriented and platform independent language. Data models are JPA entities (User, Station, Telemetry Record, Environmental Alert).
- **Framework:** Spring Boot 3.3.5 with Spring Data JPA. The backend starts on port 9090 (configurable through server.port in application.properties) and also hosts the static frontend.
- **Database:** MySQL relational database. If MySQL is not available, the application automatically falls back to an in-memory H2 database.
- **Hash Algorithm:** SHA-256 with a random salt per user, which gives a 256-bit (64 hexadecimal characters) one-way digest.
- **Simulation Heartbeat:** a @Scheduled background task adds small live variations to the telemetry of every station.
- **API Style:** 13 REST endpoints grouped into Auth, Stations, Telemetry, Simulation and Alerts. CORS is enabled so the frontend can also run standalone.

2.3 Team Structure & Module Ownership

The project is developed by a five-member team. The role and core module ownership of each member is given below.

Team Member	Assigned Role	Core Module Ownership
Devansh Joshi	Backend Services & Geospatial Algorithms	Implement GPS location pairing and the mathematical distance formula to find the nearest station.
Devansh Mittal	Backend Simulation & Automated Alerting	Build the "What-If" disaster simulation engine and automated hazard warning system.

Team Member	Assigned Role	Core Module Ownership
Dhruv Jain	Frontend UI/UX & Web Dashboard	UI design, the command center interface, live sensor cards, and real-time graphs.
Garv Kumar	Database Architecture & Application	Design the database tables, sample data seeding, and secure user login system.
Ishita Sinha	Frontend Canvas & Computer Graphics	Build the interactive 2.5D visual map and moving atmospheric particle engine on the canvas.

All team members collaborated to integrate their respective modules into a unified system and resolve technical issues during development. Regular coordination and testing helped maintain consistency between the frontend, backend, database, and simulation components.

3. DESIGN & METHODOLOGY

3.1 System Architecture

AETHERIS follows a clean four-layer architecture. The **presentation layer** (HTML, CSS and JavaScript) shows the landing page, the citizen dashboard and the digital twin canvas. The **controller layer** is made of Spring Boot REST controllers (Auth, Station, Telemetry, Simulation and Alert), with CORS configuration and role checks. The **business logic layer** holds the services: authentication, station pairing with the Haversine formula, telemetry handling, hazard injection and the scheduled heartbeat. The **data layer** uses JPA entities, Spring Data repositories and MySQL (or H2).

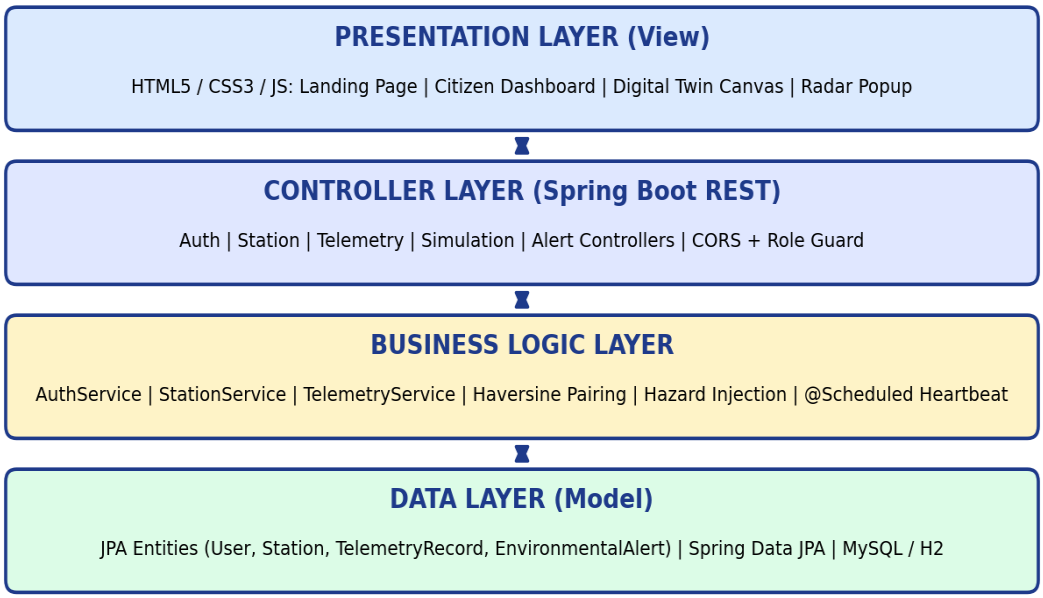


Figure 3.1: System Architecture (layers)

The frontend files (app.js, auth.js, location.js, twin-canvas.js and charts.js) call the REST API and poll live telemetry. The same files are copied into the static folder of Spring Boot, so one command starts both backend and frontend.

3.2 Geodesic Pairing Algorithm (Haversine)

When a user pairs a physical device, the browser GPS (or a state-wise preset) gives the latitude and longitude of the user. The system finds the nearest sensor station using the Haversine formula,

which gives the great-circle distance between two points on a sphere.

- $a = \sin^2(\Delta\varphi / 2) + \cos(\varphi 1) \times \cos(\varphi 2) \times \sin^2(\Delta\lambda / 2)$
- $c = 2 \times \text{atan2}(\sqrt{a}, \sqrt{(1 - a)})$
- Distance $d = R \times c$, where $R = 6371$ km (mean radius of the Earth).
- Bearing $\theta = \text{atan2}(\sin \Delta\lambda \times \cos \varphi 2, \cos \varphi 1 \times \sin \varphi 2 - \sin \varphi 1 \times \cos \varphi 2 \times \cos \Delta\lambda)$, normalized to 0 – 360 degrees.
- The station with the smallest distance is returned as the nearest station together with distance, bearing and an 8-point compass label (N, NE, E, SE, S, SW, W, NW).

The radar popup then draws a laser pairing beam across the digital twin canvas from the user position to the paired station.

3.3 AQI Verdicts and Clean Air Score

The citizen dashboard converts the technical AQI into a simple 0 to 100 **Clean Air Score** (higher is cleaner) and a plain-English verdict. The AQI levels follow the EPA scale, which is grouped into four citizen verdicts.

AQI Range	EPA Category	Citizen Verdict	Advice Style
0 – 50	Good	Crisp & Pristine	Ideal for sports, jogging and open windows
51 – 100	Moderate	Moderate	Acceptable; sensitive people take care

AQI Range	EPA Category	Citizen Verdict	Advice Style
101 – 200	Unhealthy for Sensitive Groups / Unhealthy	Sensitive Warning	Limit outdoor exercise, use masks if needed
201 and above	Very Unhealthy / Hazardous	Hazardous	Stay indoors, keep windows closed

- Clean Air Score = 100 − (AQI / 5), where the AQI is limited between 0 and 500, so the score is always between 0 and 100.
- Temperature unit toggle: °F = (°C × 9 / 5) + 32.
- The dashboard also shows six metric cards: Air Purity, Real-Feel Temperature, Solar UV, Natural Breeze, Acoustic Serenity and Atmospheric Freshness.

3.4 Edge-Case Handling

Situation	Rule
Latitude outside −90 to +90 or longitude outside −180 to +180	Coordinates are rejected as invalid, no distance is calculated.
Location permission denied by the user	The radar popup falls back to state-wise preset locations.
No stations registered	Nearest station query returns an empty result safely.
AQI value above 500	Value is clamped to 500, so Clean Air Score becomes 0.
Wrong username or password	Login is rejected, no token is issued.

Situation	Rule
Duplicate username or e-mail during registration	Registration is rejected.
OPERATOR / ADMIN action without a valid token	Request is blocked (add node, inject scenario).
MySQL not reachable at start-up	Application falls back to in-memory H2 database.

3.5 Authentication & Password Hashing

The system supports login, registration and a profile endpoint. A user may log in with either the username or the e-mail. Passwords are never stored in plain text. Each user gets a random salt, and the stored value is:

Password Hash = SHA256 (Salt + Password)

- The hash is one-way, so the original password cannot be found back from the stored hash.
- A different random salt per user means two users with the same password get different hashes.
- During login the hash is recalculated and compared in constant time.

Three roles are used for access control:

Role	Access Level
ADMIN	Full administrative control: add sensor nodes, inject simulation scenarios, acknowledge alerts.
OPERATOR	Scenario simulations and alert acknowledgements.
RESEARCHER	Read access to live telemetry, history and reports.

3.6 Hazard Injection Engine (Scenario State Machine)

The "What-If" engine lets an authorized user push all twin nodes into a hazard condition and observe how the environment responds. Only the scenarios below are accepted; any other value is rejected.

Scenario	Effect on the Digital Twin
HEATWAVE	Extreme temperature surge and ozone formation.
INDUSTRIAL_EMISSION	Particulate PM2.5 and CO2 surge.
RAIN_CLEANSING	Rainfall washes out particulates and cools the ambient air.
NORMAL	Reset Baseline: all twin nodes return to nominal conditions.

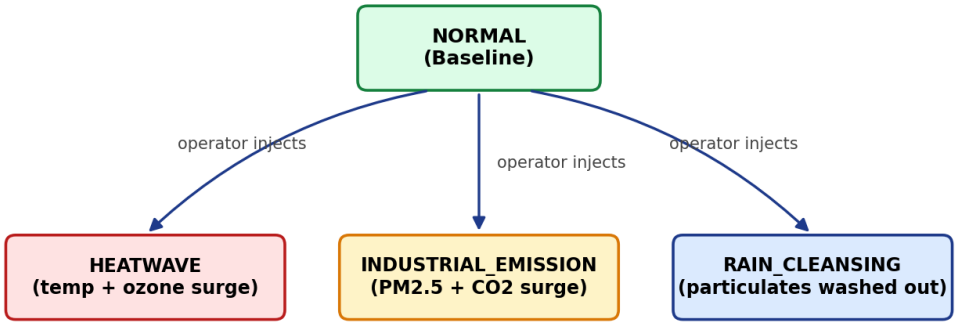


Figure 3.2: Hazard Scenario State Diagram

Alerts are created when a threshold is crossed, and they can be listed (all or only unacknowledged) and acknowledged through the Alert API.

3.7 Database Design

The database schema is provided in database/schema.sql, with seed data. It contains the following main tables. Telemetry and alerts are linked to a station through a foreign key.

Table	Main Fields
users	userId (PK), username, email, passwordHash, salt, role, lastLat, lastLng
stations	stationId (PK), name, latitude, longitude, elevation, status
telemetry_records	recordId (PK), stationId (FK), aqi, pm25, pm10, temperature, humidity, co2, noise, uv, wind, timestamp
environmental_alerts	alertId (PK), stationId (FK), severity, message, acknowledged, createdAt

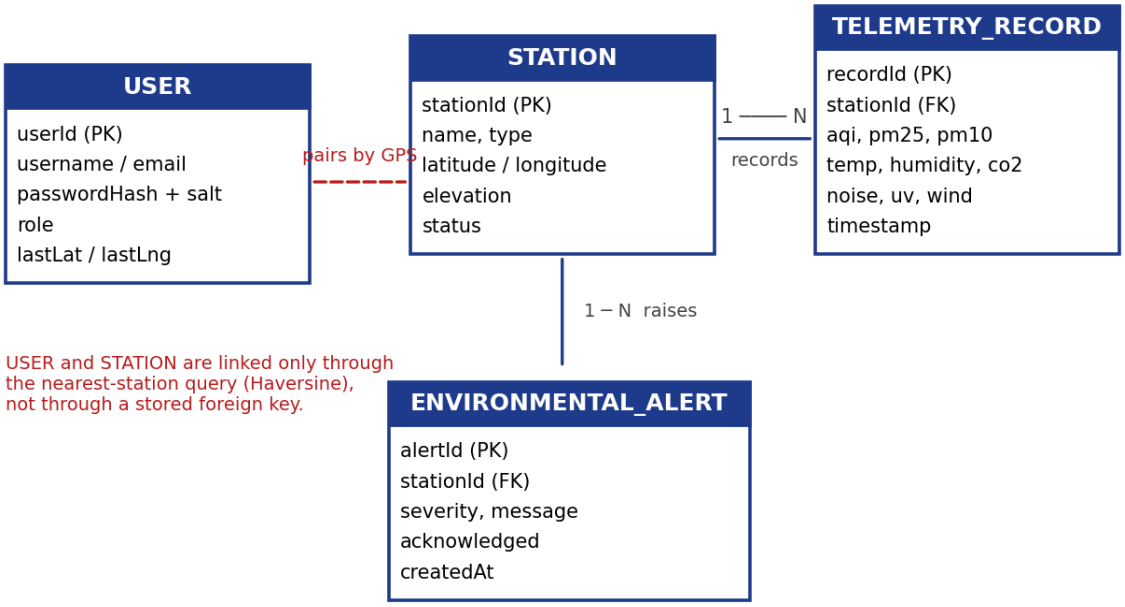


Figure 3.3: Entity Relationship Diagram

3.8 REST API Reference

Method	Endpoint	Description
POST	/api/auth/login	Authenticate with username / password, returns token
POST	/api/auth/register	Register a new user account
GET	/api/auth/me	Fetch the authenticated user profile
POST	/api/auth/update-location	Update the user device coordinates
GET	/api/stations	List all sensor stations
GET	/api/stations/{id}	Get station details by ID
POST	/api/stations	Register a new physical / virtual sensor node
GET	/api/stations/nearest?lat=&lng=	Haversine distance, bearing and nearest station
GET	/api/telemetry/live	Live telemetry across all stations
GET	/api/telemetry/history/{stationId}	Historical time-series logs
POST	/api/simulation/trigger	Inject scenario (HEATWAVE, INDUSTRIAL_EMISSION, RAIN_CLEANSING, NORMAL)
GET	/api/alerts	Threshold alerts (?unacknowledgedOnly=true)
POST	/api/alerts/{id}/acknowledge	Acknowledge an active hazard alert

4. RESULT AND DISCUSSION

4.1 System Performance

Processing Test	Observed Result	Status
Haversine: Jaipur (26.9124, 75.7873) to Delhi (28.6139, 77.2090)	Distance 235.29 km, bearing 36.15° (NE)	PASSED
Haversine: same point twice	Distance 0.00 km	PASSED
Haversine: 0.001° latitude shift	Distance ≈ 111.19 m	PASSED
AQI verdict at 50, 51, 100, 101, 200, 201	Crisp & Pristine, Moderate, Moderate, Sensitive Warning, Sensitive Warning, Hazardous	PASSED
Clean Air Score at AQI 25, 150, 500	95, 70 and 0	PASSED
Temperature unit conversion 0, 37, 100 °C	32 °F, 98.6 °F, 212 °F	PASSED
Password hash with two random salts (same password)	Different 64-character hashes; correct password verifies, wrong password fails	PASSED

4.2 Project Screenshots

The following figures show the working screens of the AETHERIS application. Each screen is shown in its own frame.



Figure 4.1: Product Showcase Landing Page

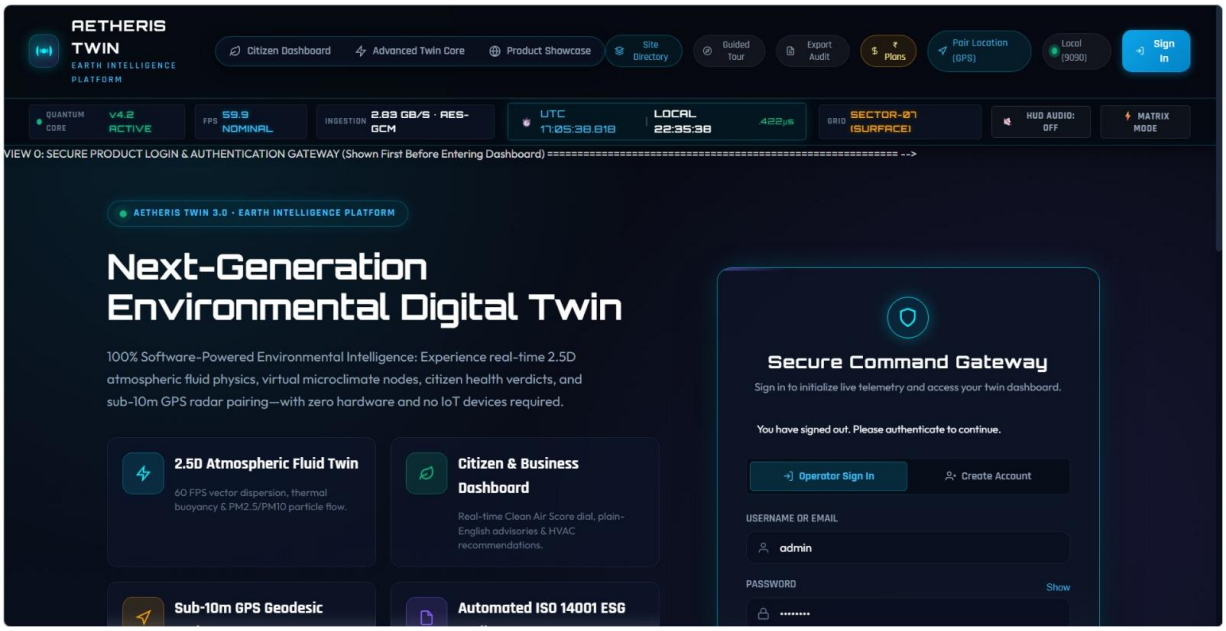


Figure 4.2: Login and Register Modal

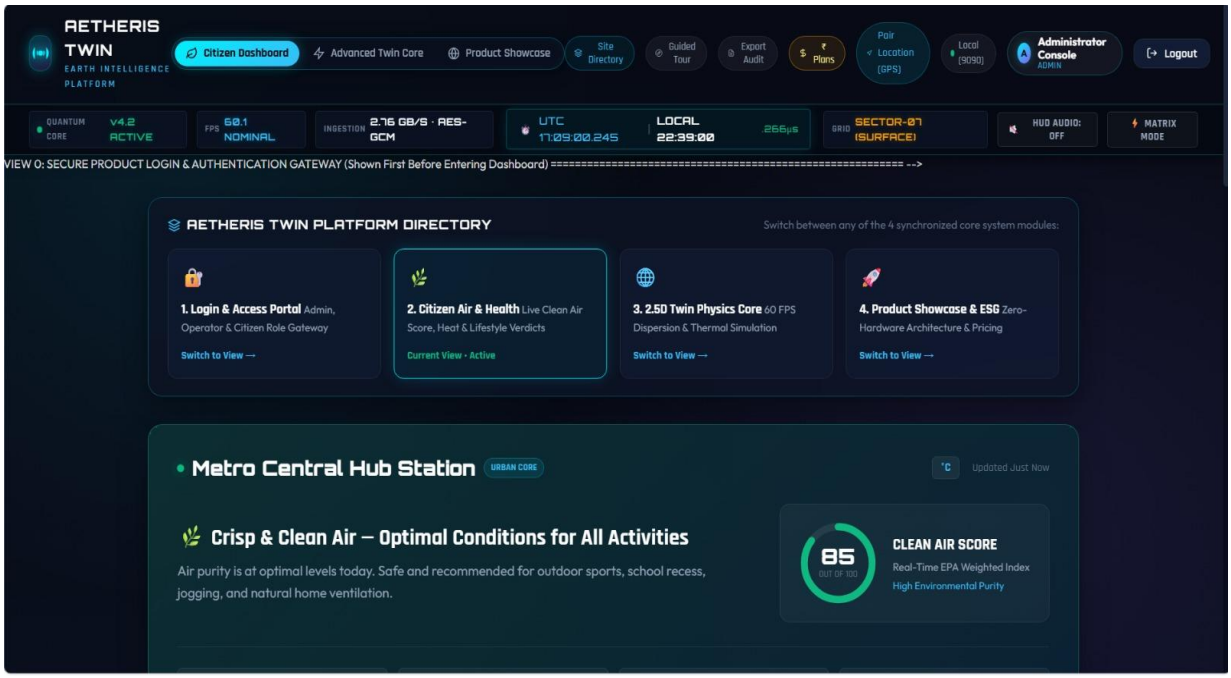


Figure 4.3: Citizen Dashboard (Clean Air Score and Verdict)



Figure 4.4: 2.5D Digital Twin Canvas (Air Dispersion Mode)

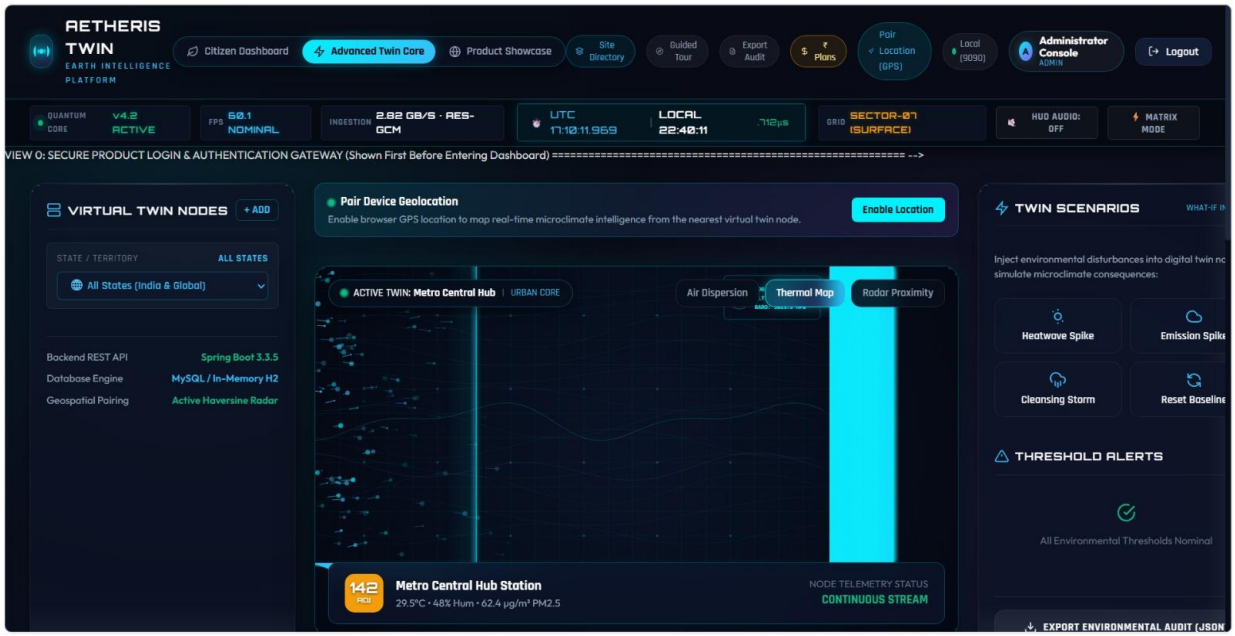


Figure 4.5: GPS Geolocation Pairing Radar

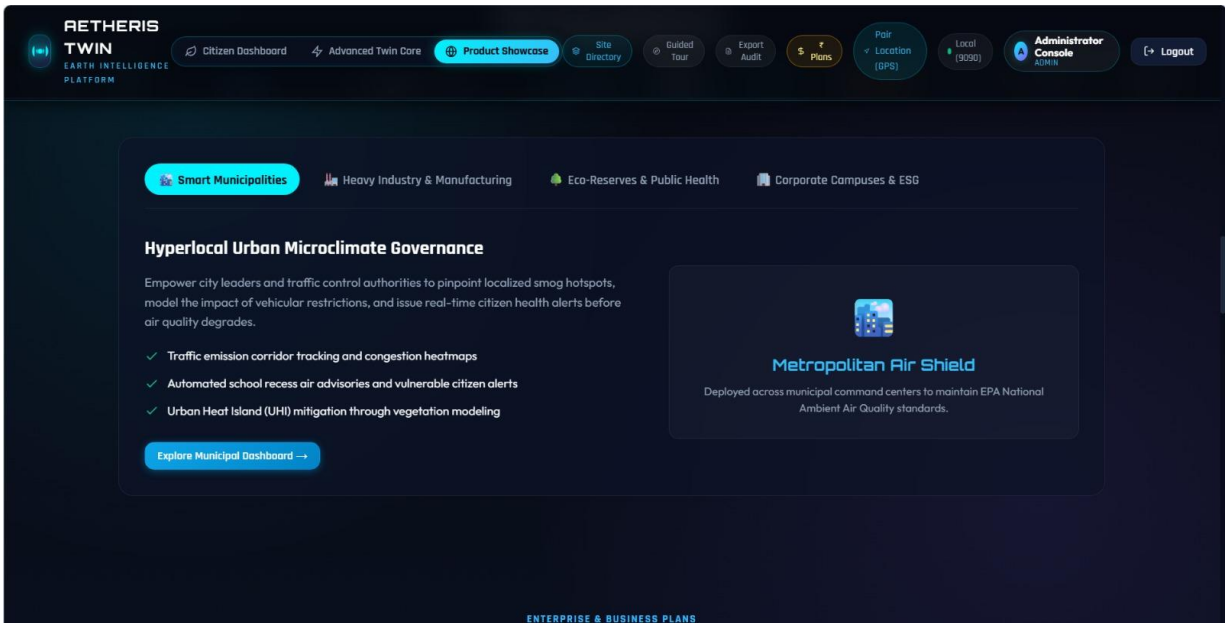


Figure 4.6: Smart Municipalities and Environmental Solutions

4.3 Algorithm & Security Reliability

The geolocation, scoring and authentication logic was checked with the following validation cases. The results match the expected output of the formulas and rules.

Case	Input	Expected Behaviour	Result
1	Valid lat / lng with several stations	Nearest station, distance (km) and bearing returned	Accepted
2	Latitude = 95	Coordinates rejected as invalid	Rejected
3	Longitude = −200	Coordinates rejected as invalid	Rejected
4	Wrong password for an existing user	No token issued	Rejected
5	Scenario injection without a valid token	Request blocked	Rejected
6	AQI = 650	Clamped to 500, Clean Air Score = 0, verdict Hazardous	Accepted

Passwords are stored only as salted SHA-256 hashes, so the original password cannot be recovered from the database. Sensitive operations such as adding nodes and injecting scenarios are available only to authorized roles. Constant-time hash comparison is used during password verification to help reduce timing-based attacks. Invalid inputs and unauthorized requests are rejected to maintain data integrity and prevent unauthorized access.

4.4 Limitations Observed

- The accuracy of GPS pairing depends on the user's device and environment. Sub-10 m accuracy is possible only with a good GPS signal.
- By default the telemetry is produced by the built-in simulation heartbeat. Real hardware sensors must be connected through the station API to get real readings.

- Salted SHA-256 is acceptable for a lab project, but production systems should use a slow password hashing algorithm such as bcrypt or Argon2.
- The demo credentials (admin / admin123 and operator / operator123) are meant only for demonstration and must be changed before real use.
- The in-memory H2 fallback loses all data when the application stops, so MySQL should be used for permanent storage.
- The 2.5D canvas engine needs a reasonably modern browser and may run slowly on very old devices.
- The environmental simulation provides indicative results and should not be treated as a substitute for verified measurements from physical sensors.
- The accuracy of hazard scenarios depends on the predefined simulation rules and may not fully represent real-world environmental conditions.
- Network connectivity and backend availability can affect data retrieval, authentication, and dashboard updates.
- The system's performance with a large number of stations and telemetry records requires further testing under realistic workloads.

5. CONCLUSION AND RECOMMENDATION

5.1 Conclusion

AETHERIS successfully demonstrates how Advance Java technologies (Java 17, Spring Boot, Spring Data JPA, REST and MySQL) can be combined with an interactive HTML5 Canvas frontend to build a complete digital twin of an environment. The system gives citizens a simple Clean Air Score with clear health advice, gives operators a safe "what-if" engine to test hazard scenarios, and gives organizations an audit-ready compliance report.

5.2 Key Achievements

- Built a Spring Boot 3.3.5 REST backend with 13 endpoints, JPA persistence and MySQL with automatic H2 fallback.
- Developed a 2.5D Canvas digital twin with wind currents, particulate dispersion and three visualization modes.
- Implemented Haversine based GPS pairing with distance, compass bearing and a live pairing beam.
- Created a Citizen Dashboard with an animated Clean Air Score and dynamic health verdicts.
- Implemented salted SHA-256 authentication with ADMIN, OPERATOR and RESEARCHER roles.
- Built a hazard injection engine, an interactive guided tour and a one-click certified audit report.

5.3 Recommendations for Future Scope

- **Real IoT Integration:** connect real hardware sensors using MQTT or HTTP gateways instead of only simulated data.

- **Forecasting:** use machine learning to predict AQI and temperature trends for the next hours.
- **Stronger Security:** move to bcrypt / Argon2 password hashing and JWT with refresh tokens.
- **Notifications:** send e-mail or SMS alerts when a hazard threshold is crossed.
- **Mobile Application:** build an Android and iOS app for citizens.
- **GIS-Based Environmental Mapping:** display monitoring stations and pollution hotspots on interactive maps.
- **Advanced Digital Twin Visualization:** improve 2.5D visualization with dynamic pollutant dispersion and realistic environmental effects.
- **Historical Data Analytics:** analyze historical environmental data to identify pollution patterns and seasonal variations.
- **Anomaly Detection:** identify unusual environmental readings and sudden changes in AQI levels.
- **Multi-Station Comparison:** compare AQI, temperature, humidity, and other parameters across multiple monitoring stations.

6. APPENDICES

Appendix A: System Diagrams

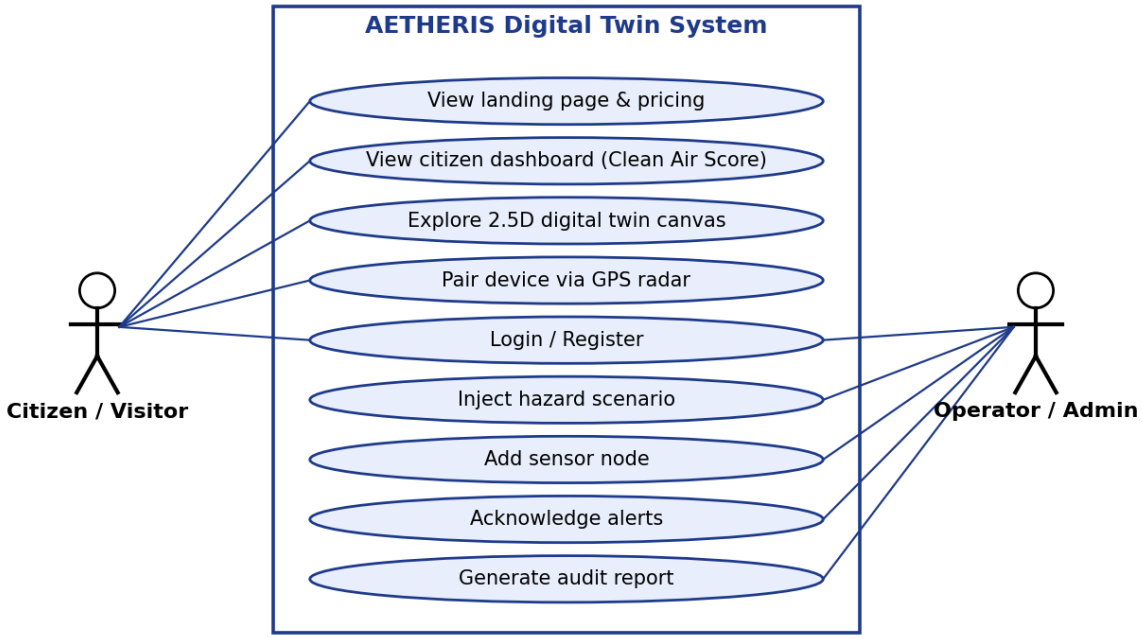


Figure 6.1: Use Case Diagram



Figure 6.2: Data Flow Diagram – Level 0

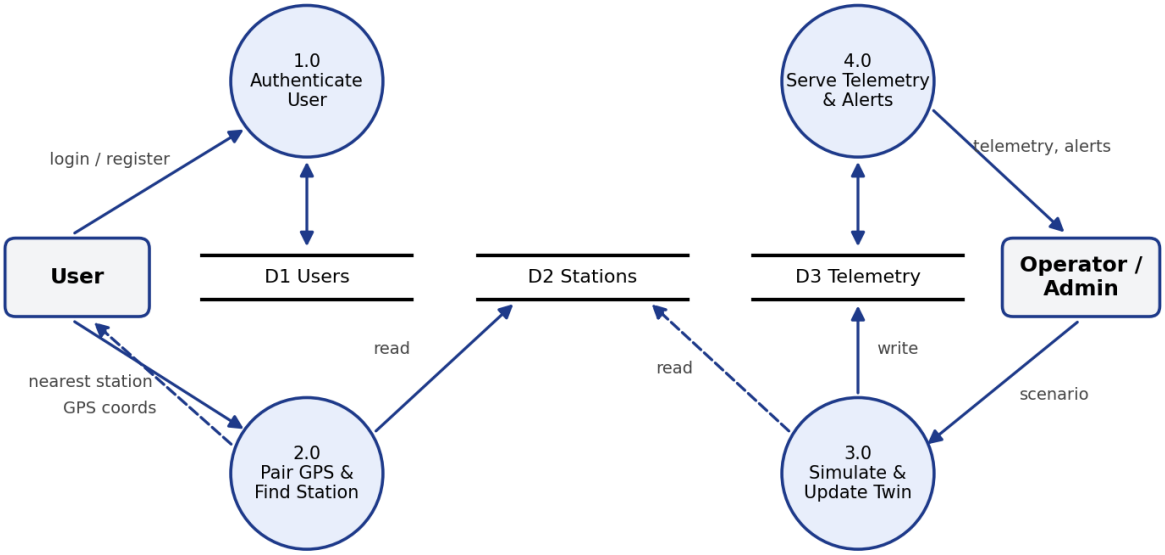


Figure 6.3: Data Flow Diagram – Level 1

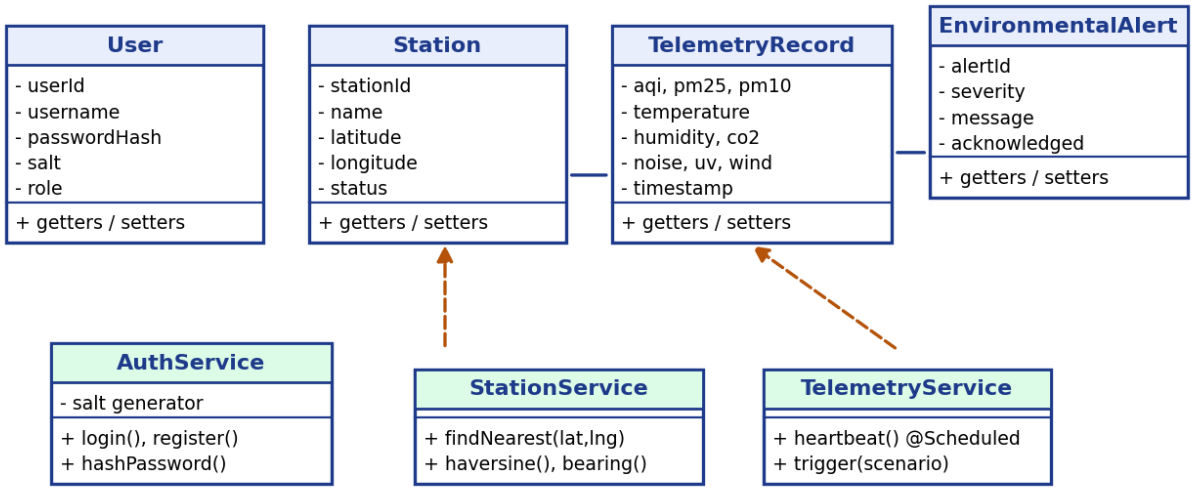


Figure 6.4: Class Diagram

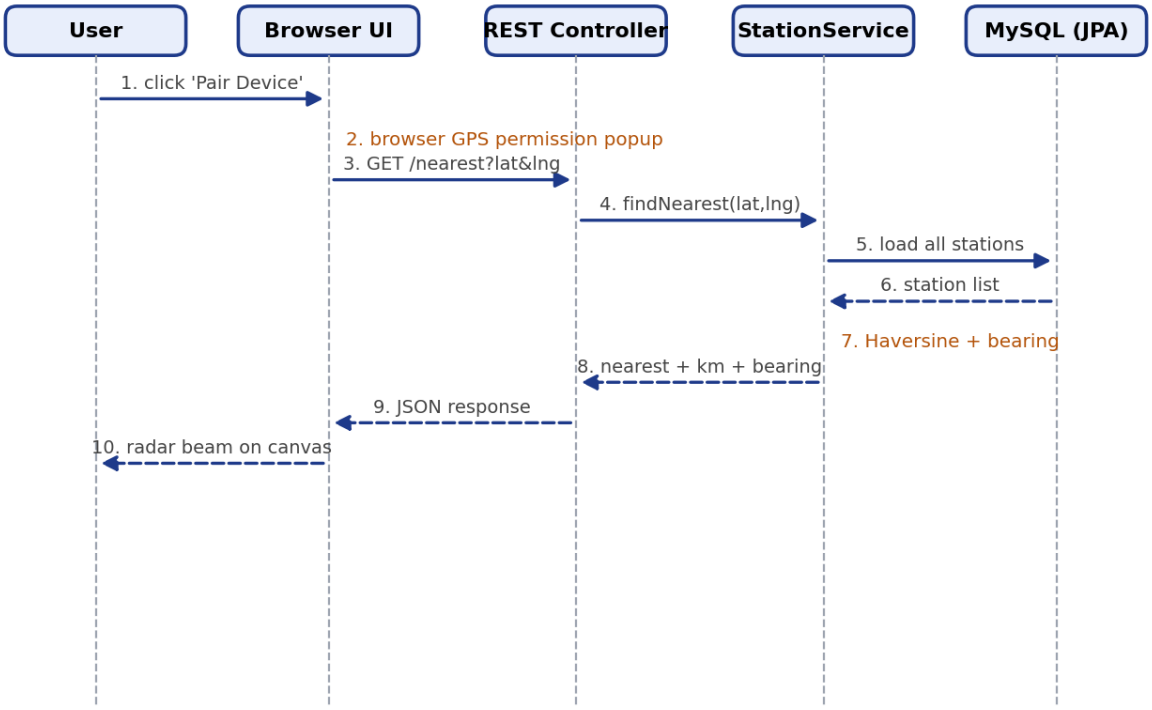


Figure 6.5: Sequence Diagram – GPS Pairing of a Device

Appendix B: Core Source Code (Java)

The listings below show the core logic of the system: geodesic calculation, password hashing, AQI verdicts and the simulation scenarios.

GeoUtils.java – Haversine distance and bearing

```
public final class GeoUtils {

    private static final double EARTH_RADIUS_KM = 6371.0;
    private GeoUtils() { }

    public static double distanceKm(double lat1, double lng1,
                                    double lat2, double lng2)
{
        double dLat = Math.toRadians(lat2 - lat1);
        double dLng = Math.toRadians(lng2 - lng1);
        double a = Math.sin(dLat / 2) * Math.sin(dLat / 2)
                 + Math.cos(Math.toRadians(lat1)) *
Math.cos(Math.toRadians(lat2))
                 * Math.sin(dLng / 2) * Math.sin(dLng / 2);
        double c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 -
a));
        return EARTH_RADIUS_KM * c;
    }

    public static double bearingDeg(double lat1, double lng1,
                                    double lat2, double lng2)
{
        double p1 = Math.toRadians(lat1), p2 =
Math.toRadians(lat2);
        double dl = Math.toRadians(lng2 - lng1);
        double y = Math.sin(dl) * Math.cos(p2);
        double x = Math.cos(p1) * Math.sin(p2)
                 - Math.sin(p1) * Math.cos(p2) * Math.cos(dl);
        return (Math.toDegrees(Math.atan2(y, x)) + 360.0) %
360.0;
    }
     public static String compass(double bearing) {
        String[] dir = {"N", "NE", "E", "SE", "S", "SW", "W",
"NW"};
        return dir[(int) Math.round(bearing / 45.0) % 8];
    }
```

```
    public static boolean isValid(double lat, double lng) {
        return lat >= -90 && lat <= 90 && lng >= -180 && lng
<= 180;
    }
}
```

Geo Utils calculates the great-circle distance between two geographic locations using the Haversine formula. It also calculates the initial compass bearing and converts it into one of eight directions, such as N, NE, E, or SW. The coordinate validation method checks whether the latitude and longitude values are within valid ranges.

PasswordUtil.java – Salted SHA-256 password hashing

```
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.security.SecureRandom;

public final class PasswordUtil {

    private static final SecureRandom RANDOM = new
SecureRandom();

    private PasswordUtil() { }

    // Random 16-byte salt, stored in hex form next to the
hash
    public static String newSalt() {
        byte[] salt = new byte[16];
        RANDOM.nextBytes(salt);
        return toHex(salt);
    }

    // hash = SHA256( salt + password )  ->  64 hexadecimal
characters
    public static String hash(String password, String salt) {
        try {
            MessageDigest md = MessageDigest.getInstance("SHA-
256");
```

```
            byte[] out = md.digest((salt +
password).getBytes(StandardCharsets.UTF_8));
            return toHex(out);
        } catch (NoSuchAlgorithmException e) {
            throw new IllegalStateException("SHA-256 not
available", e);
        }
    }

    // Constant-time comparison to avoid timing attacks
    public static boolean verify(String password, String salt,
String storedHash) {
        byte[] a = hash(password,
salt).getBytes(StandardCharsets.UTF_8);
        byte[] b =
storedHash.getBytes(StandardCharsets.UTF_8);
        return MessageDigest.isEqual(a, b);
    }

    private static String toHex(byte[] bytes) {
        StringBuilder sb = new StringBuilder();
        for (byte b : bytes) {
            sb.append(String.format("%02x", b));
        }
        return sb.toString();
    }
}
```

Password Util protects passwords by generating a random salt and combining it with the password before hashing. During login, it generates the hash again and compares it with the stored hash. The constant-time comparison helps reduce timing-based information leakage.

Aqi Util.java – Verdict and Clean Air Score

```
public final class AqiUtil {

    private AqiUtil() { }

    // Plain-English verdict shown on the Citizen Dashboard
    public static String verdict(int aqi) {
```

```
        if (aqi <= 50)  return "Crisp & Pristine";
        if (aqi <= 100) return "Moderate";
        if (aqi <= 200) return "Sensitive Warning";
        return "Hazardous";
    }

    // Clean Air Score (0-100): higher is cleaner, AQI above
500 is clamped
    public static int cleanAirScore(int aqi) {
        int bounded = Math.max(0, Math.min(aqi, 500));
        return (int) Math.round(100.0 - bounded / 5.0);
    }

    // Unit toggle used by the KPI gauges
    public static double toFahrenheit(double celsius) {
        return celsius * 9.0 / 5.0 + 32.0;}}
```

Aqi Util converts AQI values into simple environmental verdicts for the Citizen Dashboard. It calculates a Clean Air Score between 0 and 100, where a higher score indicates cleaner air. It also converts temperatures from Celsius to Fahrenheit for dashboard display.

7. REFERENCES

[1] Spring Boot Reference Documentation (version 3.3). Available at: https://docs.spring.io/spring-boot/

[2] Spring Data JPA Reference Documentation. Available at: https://spring.io/projects/spring-data-jpa

[3] Oracle, Java SE API Documentation: Message Digest. Available at: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/security/MessageDigest.html

[4] NIST, FIPS PUB 180-4: Secure Hash Standard (SHA-256). Available at: https://csrc.nist.gov/publications/detail/fips/180/4/final

[5] MySQL Reference Manual. Available at: https://dev.mysql.com/doc/

[6] MDN Web Docs: Canvas API and Geolocation API. Available at: https://developer.mozilla.org/

[7] U.S. EPA, Technical Assistance Document for the Reporting of Daily Air Quality – the Air Quality Index (AQI). Available at: https://www.airnow.gov/aqi/

[8] R. W. Sinnott, "Virtues of the Haversine", Sky and Telescope, vol. 68, no. 2, 1984.

[9] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.